

# Real World Automating To Support Testing

OdinSTAR 2017

Alan Richardson

- [www.eviltester.com](http://www.eviltester.com)
- [www.compendiumdev.co.uk](http://www.compendiumdev.co.uk)
- [@eviltester](https://twitter.com/eviltester)

# Synopsis

Very often we talk about "Test Automation" when we should be talking about how we automate to support our test strategy and testing process. We can support our strategy and process by tactically automating tasks or by adopting a more strategic approach to our automated execution.

In this talk we will explore the difference between automating tactically and strategically so you don't get fooled by tactics that look like strategy. We will also cover concepts that help you create robust and easy to maintain automated solutions, and the techniques to implement those concepts, to support your all testing: regression, functional, performance, end to end, and exploratory.

# What is Test Automation?

- There is no such thing as Test Automation
- You can't automate testing
- Testing vs Checking
- Tools

**The Testing Industry finds "Test Automation" contentious**

**We automate our activities to support  
our work**

**People have always automated stuff**



**Movable Tower.**

# What do we automate?

- not about "test automation"
- automate tasks
- automate the assertions used to check that the stories/requirements are still 'done'

We reuse this to automate 'tests' to support our test process

# An Example "This is an automated test?"

```
@Test
public void createPreviewMVP() throws IOException {

    String args[] = {
        new File(System.getProperty("user.dir"),
            "pandocifier.properties").getAbsolutePath()};

    new PandocifierCLI().main(args);

    String propertyFileName = args[0];
    File propertyFile = new File(propertyFileName);
    PandocifierConfig config;
    config = new PandocifierConfigFileReader().
        fromPropertyFile(propertyFile);

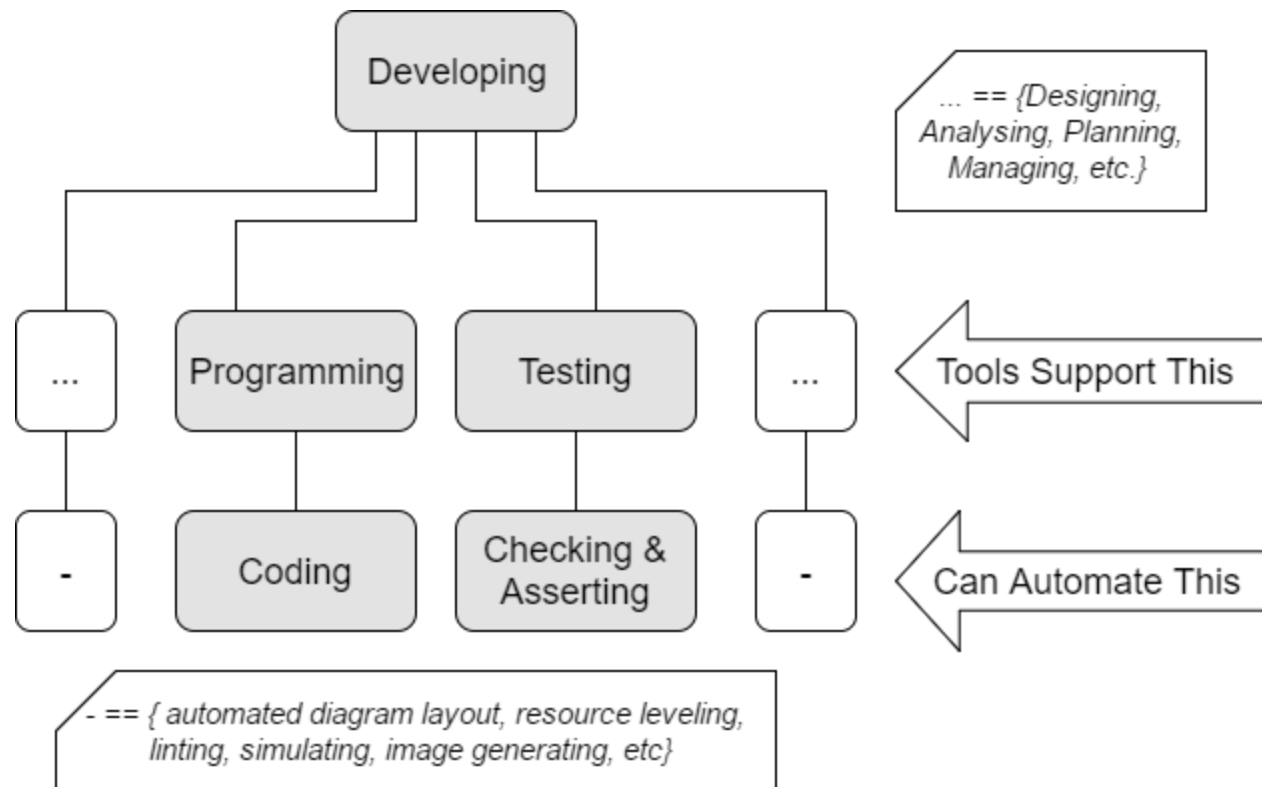
    Assert.assertTrue(new File(
        config.getPreviewFileName()).exists());
}
```

# An Example "Ce n'est pas un test"

- This is a Test because it says it is a `@Test`
- This is an Automated Test because it **checks** something (file exists)
- This is an Automated Test because it **Asserts** on the check
- This is not an Application - it doesn't have a GUI or a standalone execution build
- This is a tool to support my work - I use it to build one of my books

This is **automated execution of a process**

# We Automate Parts of our Development Process: Coding, Checking, Asserting, ...



# Real World Automation

A world of:

- Best Practices that are no such thing
- Magic Tools that Automatically create and maintain Tests
- 'Code-Free' Programming
- Capture Replay
- Throw Away Automation and start again
- ROI
- ... and more

# (The) 5 Secrets of Successful Real World Automating To Support Testing

- Get work done
  - *now vs longer term* (tactically vs strategically)
- We automate Flows
- Synchronisation
- Multiple Abstractions
- Abstract to libraries, not frameworks

# Tactical Reality - I don't really care, I just want to get work done

Suggests we also have 'strategic reality'

# Automating Tactically vs Strategically

| What is Tactical?   | What is Strategic?        |
|---------------------|---------------------------|
| Solve a problem     | Agreed & Understood Goals |
| Short Term          | Long Term                 |
| Change as necessary | Slower to Change          |
| Your Time           | 'project' time            |

# Warning - sometimes tactics look like strategy

- Use of Jenkins is not Continuous Integration
- Automated Deployments are not Continuous Delivery
- Containers are not always good environment provision
- Page Objects are not guaranteed good system abstractions

# You know you are Automating strategically when...

- Reduced Maintenance
- Reduced change
- Everyone agrees why
- No fighting for 'time'
- It's not about roles
- It gets done

# Myths of Real World Automating

- Application must be stable
- Non-technical staff need text based tools
  - Gherkin
  - Keyword Driven

# Abstraction

"The purpose of abstraction is not to be vague, but to create a new semantic level in which one can be absolutely precise."

- *E. W. Dijkstra*
- 1972 ACM Turing Lecture: '[The Humble Programmer](#)'

# How do we Automate?

- flows through the system
- vary data
- abstract the execution

# Automating Strategically

Abstractions - "modeling the system in code"

- abstractions are a big part of automating strategically
- abstractions are about modelling
  - people seem to be scared of having too many models

*see also "Domain Driven Design"*

# No Abstractions - Web Testing

```
@Test
public void canCreateAToDoWithNoAbstraction(){
    driver = new FirefoxDriver();
    driver.get(
        "http://todomvc.com/architecture-examples/backbone/");

    int originalNumberOfTodos = driver.findElements(
        By.cssSelector("ul#todo-list li")).size();

    WebElement createTodo;
    createTodo = driver.findElement(By.id("new-todo"));
    createTodo.click();
    createTodo.sendKeys("new task");
    createTodo.sendKeys(Keys.ENTER);
    assertTrue(driver.findElement(
        By.id("filters")).isDisplayed(), is(true));

    int newTodos = driver.findElements(
        By.cssSelector("ul#todo-list li")).size();
    assertTrue(newTodos, greaterThan(originalNumberOfTodos));
}
```

## But there were Abstractions

- WebDriver - abstraction of browser
- FirefoxDriver - abstraction Firefox (e.g. no version)
- WebElement - abstraction of a DOM element
- createToDo - variable - named for clarity
- Locator Abstractions via methods `findElement` `findElements`
- Manipulation Abstractions `.sendKeys`
- Locator Strategies `By.id` (did not have to write CSS Selectors)
- ...

## Lots of Abstractions

## Using Abstractions - Same flow

```
@Test
public void canCreateAToDoWithAbstraction() {
    TodoMVCUser user =
        new TodoMVCUser(driver, new TodoMVCSite());

    user.opensApplication().
        and().
        createNewToDo("new task");

    ApplicationPageFunctional page =
        new ApplicationPageFunctional(driver,
                                      new TodoMVCSite());

    assertThat(page.getCountOfToDoItems(), is(1));
    assertThat(page.isFooterVisible(), is(true));
}
```

# What Abstractions were there?

- `TodoMVCUser`
  - User
  - Intent/Action Based
  - High Level
- `ApplicationPageFunctional`
  - Structural
  - Models the rendering of the application page

# REST Test - No Abstractions

```
@Test
public void aUserCanAccessWithBasicAuthHeader(){

    given().
        contentType("text/xml").
        auth().preemptive().basic(
            TestEnvDefaults.getAdminUserName(),
            TestEnvDefaults.getAdminUserPassword()).
    expect().
        statusCode(200).
    when().
        get(TestEnvDefaults.getURL().toExternalForm() +
            TracksApiEndpoints.todos);
}
```

## But there were Abstractions

- RESTAssured - `given`, `when`, `then`, `expect`, `auth`, etc.
- RESTAssured is an abstraction layer
- Environment abstractions i.e. `TestEnvDefaults`

Lots of Abstractions

But a lack of flexibility

# Different levels if abstractions increase flexibility

```
@Test
public void aUserCanAuthenticateAndUseAPIWithBasicAuth(){

    HttpMessageSender http = new HttpMessageSender(
        TestEnvDefaults.getURL());

    http.basicAuth( TestEnvDefaults.getAdminUserName(),
        TestEnvDefaults.getAdminUserPassword());

    Response response = http.getResponseFrom(
        TracksApiEndpoints.todos);

    Assert.assertEquals(200, response.getStatusCode());
}
```

# Supports

- regular automated execution with assertions
- creation and maintenance by many roles
- exploratory testing
- model based automated execution
- performance testing
  - multithreaded bots
  - potential re-use in performance tools

# Law of Requisite Variety

"only variety can destroy variety"

- W. Ross Ashby

"only variety can absorb variety"

- Stafford Beer

*see also Attenuation and Amplification.*

# Attenuation and Amplification

- Attenuation can 'Destroy' variety
  - because you are not made aware of it
- Amplification can 'Destroy' variety
  - because it obscures the variety

Skill Sets and flexibility allow you to absorb and handle the variety of requests thrown at you and your architecture for supporting automating.

# Frameworks can 'Destroy' variety

- Frameworks force us to comply with their structure
  - or things get 'hard'
  - or we don't reap the benefits from the Framework
- Some things are not possible
- Some things never occur to you to do

# Models

## We have a lot of models

- user intent model - what I want to achieve
- user action model - how I do that
- interface messaging model - how the system implements that
- admin models, support models, GUI models

## And models overlap

- user can use the API or the GUI to do the same things

# Modelling is a key 'testing' skill

- understand the system from different perspectives
- technical models
- flow models
- structural models
- analyse for risk
- analyse for *coverage*

# Abstract with different implementations

- 'interfaces'
- different implementations do the same thing 'dependency injection'

```
user = new User().using(new API_implementation())  
user = new User().using(new GUI_implementation())
```

@Test usage unchanged

```
project = user.createNewProject("my new project")  
user.addTaskToProject("my first task", project)
```

# Abstractions are DSL to support the person writing and maintaining and reviewing the test

```
user.addTaskToProject("my first task", project)
```

we can change them to make `@Test` code more readable

```
user.forProject(project).addTask("my first task")
```

# Abstractions can make developers nervous

- too much code
- too many layers

**But... code to support our test process  
needs to handle more variety than code  
to support system requirements**

# When we don't do this

- test code takes a long time to maintain
- test code takes too long to write
- test code is hard to understand
- test code is brittle - "every time we change the app the tests break/ we have to change test code"
- execution is flaky - intermittent test runs - "try running the tests again"

# Flaky - often implies poor synchronization

- Understand if your 'framework' is synchronising for you
- Frameworks do not sync on application state, they sync on 'framework' state
- Synchronise on minimum application state
  - e.g.
    - when page loads, wait till it is ready
    - then all other actions require no synchronisation
    - if Ajax involved then wait for stable Dom

# Keyword and Gherkin Abstractions

# Writing `@Test` DSL code means that

- test code is written at the level of the test
- it is easy to write
- it is easy to understand
- the abstraction layers have to be maintained when the application changes
- the `@Test` code only changes when the test scenario changes
- could re-use `@Test` at the API and at the GUI and in between

# Tactical Approach on Agile projects

- exploratory testing to get the story to done
- create a tech debt backlog for 'missing' automated tests
- `@Test` code is tactical and we can knock it up (not as good as production)

# Strategic Approach on Agile Projects

- automate the checking of acceptance criteria to get the story to done
- ongoing execution of the automated assertions for the life of the story
- implies
  - maintain this for the life of the story in the application
  - deleting the @Test code when no longer relevant
  - amending code structure to make relevant as stories change and evolve
  - @Test code as strategic asset (just like production code)

# Abstractions support different types of testing

- exploratory testing
- stress/performance testing (bot example) requires thread safe abstractions

**Good abstractions encourage creativity, they do not force compliance and restrict flexibility.**

# Successful Real World Automating

- Automate Tactically to solve problems
- Automate Strategically for Projects
- ongoing assertion checking automated in `@Test` code
- Abstractions model the system in code
- Write abstractions at the domain level of the assertion concern
- Good abstractions can be used to support: exploratory testing, performance, adhoc automating

# Learn to "Be Evil"

- [www.eviltester.com](http://www.eviltester.com)
- [@eviltester](#)
- [www.youtube.com/user/EviltesterVideos](http://www.youtube.com/user/EviltesterVideos)

# Learn About Alan Richardson

- [www.compendiumdev.co.uk](http://www.compendiumdev.co.uk)
- [uk.linkedin.com/in/eviltester](http://uk.linkedin.com/in/eviltester)

# Follow

- LinkedIn - [@eviltester](#)
- Twitter - [@eviltester](#)
- Instagram - [@eviltester](#)
- Facebook - [@eviltester](#)
- Youtube - [EvilTesterVideos](#)
- Pinterest - [@eviltester](#)
- Github - [@eviltester](#)
- Slideshare - [@eviltester](#)

# BIO

Alan is a test consultant who enjoys testing at a technical level using techniques from psychotherapy and computer science. In his spare time Alan is currently programming a [multi-user text adventure game](#) and some [buggy JavaScript games](#) in the style of the Cascade Cassette 50. Alan is the author of the books "[Dear Evil Tester](#)", "[Java For Testers](#)" and "[Automating and Testing a REST API](#)". Alan's main website is [compendiumdev.co.uk](http://compendiumdev.co.uk) and he blogs at [blog.eviltester.com](http://blog.eviltester.com)